# PROGRAMMING IN C++

Made By : **KASHIF SAYYAD**

## UNIT 1 — INTRODUCTION TO C++

### ■ Structure of a C++ Program and Header Files

A C++ program is organised, like the human body, into sections: a typical structure is documented below. The **header files** are the prepackaged collections of functions and objects which we call through the preprocessor directive **#include**: **iostream** (input/output streams cin and cout), **cmath** (mathematical functions), **cstring** (string functions), **cstdlib** (general utilities), **fstream** (file handling), **iomanip** (output formatting like setw and setprecision). The older C-style headers like **stdio.h** may also be used in C++. The statement **using namespace std;** tells the compiler that we are using the standard namespace, so that we may write *cout* directly instead of *std::cout*.

```
#include         // header file for input / output
using namespace std;          // standard namespace

// global declaration section (optional)

int main()                   // every C++ program begins at main()
{
    int marks;               // local declaration

    cout << "Enter your marks : ";   // output with cout
    cin  >> marks;                   // input  with cin

    cout << "You typed : " << marks << endl;
    return 0;                // 0 tells the OS 'program finished successfully'
}
```

---

### ■ cin and cout, Basic Syntax, Variables, Data Types and Operators

**cout** is the standard output stream object (of class ostream) used together with the **insertion operator <<**, and **cin** is the standard input stream object (of class istream) used with the **extraction operator >>**; both live in the **iostream** header. The statements of a C++ program end with a semicolon and are grouped into blocks by curly braces; the comments are of two kinds: the single-line comment with // and the multi-line comment with **/* .... */**. A **variable** is a named memory location whose contents can change during execution; every variable must be **declared** with a type before use. C++ is **strongly and statically typed**: the type is checked by the compiler and fixed at compilation time.

| Data type | Meaning | Typical size / range |
|---|---|---|

| | | |
|---|---|---|
| int | whole numbers | 4 bytes, about −2,147,483,648 to 2,147,483,647 |
| float | single-precision real numbers | 4 bytes, about 7 significant digits |
| double | double-precision real numbers | 8 bytes, about 15 significant digits |
| char | one character | 1 byte, e.g. 'A', stored as its ASCII code |
| bool | Boolean | true / false |
| void | typeless: special use in functions | no value |
| long / short / unsigned modifiers | change the size or range of a type | long double, unsigned int, short int, etc. |

---

**Operators** of C++: the **arithmetic** operators (+ − * / %), **relational** operators (< > <= >= == !=), **logical** operators (&& || !), **assignment** operators (= and the compound += −= *= /= %=), **increment / decrement** (++ −−, used before or after the variable), **conditional (ternary) operator** (condition ? value1 : value2) and the bitwise operators (& | ^ << >> ~). Operator **precedence** decides which operation is done first; brackets always have the highest priority.

## ■ Control Statements

**Control statements** are of two broad kinds: **decision-making** statements (if, if ... else, nested if, else ... if ladder, switch) and **unconditional jumps** (break, continue, goto, return). The **if** statement executes its block on a true condition; the **if ... else** gives the alternative path; **switch** selects one case among many on the value of an integer or character expression, with **default** handling the remaining cases; every case ends in **break** to stop the flow through the next ones.

```
int n;
cin >> n;

if (n % 2 == 0)
    cout << "even number";
else
    cout << "odd number";

switch (n) {
    case 1:  cout << "one";  break;
    case 2:  cout << "two";  break;
    default: cout << "many";
}
```

---

## ■ Loops and Loop-Control Statements

A **loop** repeats a block while (or until) a condition stands. C++ has three loops: the **for** loop (initialisation; condition; increment all in the header — best when the count of repetitions is known), the **while** loop (condition tested before the body — an entry-controlled loop) and the **do ... while** loop (body first, condition after — an exit-controlled loop which always runs at least once). The loop-control statements change the normal flow: **break** ends the whole loop at once; **continue** skips the rest of the current round and jumps to the next one. A loop inside a loop is a nested loop; **break** comes out of the innermost loop only.

```
// table of 5 with for
for (int i = 1; i <= 10; i++)
    cout << "5 x " << i << " = " << 5 * i << endl;

// sum until user types 0 (while)
int sum = 0, x;
cin >> x;
while (x != 0) {
    sum += x;
    cin >> x;
}
cout << "sum = " << sum;

// do-while menu
do {
    cout << "1.say hello  0.quit";
    cin >> x;
} while (x != 0);
```

---

**Remember :** Difference between the loops: **while** test first and run the body zero or more times; **do ... while** runs the body first and tests last, so the body always runs at least once; **for** is only a compressed while with its initialisation, condition and increment written in the header.

# UNIT 2 — PROCEDURAL AND OBJECT-ORIENTED PROGRAMMING

## ■ POP and OOP: Concepts, Features and Differences

In **Procedural Oriented Programming (POP)** — the style of C, Pascal and FORTRAN — the program is divided into **functions**; data moves from one function to another, and the functions are the active part while the data is passive. In **Object Oriented Programming (OOP)** — the style of C++, Java and Smalltalk — the program is divided into **objects**, each combining **data (attributes)** with the **functions (methods)** that may use that data; the data of an object is protected from outside interference and the objects talk to each other through messages (function calls).

| Procedural programming (POP) | Object-oriented programming (OOP) |
|---|---|

| | |
|---|---|
| The program is divided into functions. | The program is divided into objects. |
| Top-down design approach. | Bottom-up design approach. |
| Data moves freely; it may be used by any function. | Data is hidden in classes; only member functions may touch it. |
| There is no data-hiding; data can be accidentally changed. | Encapsulation offers data security. |
| Overloading is not possible; one name for one function. | Function and operator overloading are possible. |
| Best for small straightforward programs. | Best for big, changing and reusable software. |
| C, Pascal, FORTRAN. | C++, Java, C#, Python. |

---

## ■ Classes and Objects

A **class** is a user-defined data type that binds together the data members and the member functions which operate on the data; the class is only a **blueprint** and by itself occupies no memory. An **object** is a real **instance** of the class: the class shape is cut out into a real variable which does occupy memory. The members of a class have access specifiers: **private** (default) — usable only inside the class; **public** — usable everywhere; and **protected** — usable in the class and in the derived classes. Usually data is kept private and the functions that read or change it are public; this arrangement is the practical form of data hiding.

```
class Student {
private:                          // data hiding
    int   roll;
    float marks;
public:
    void getData() {              // member function
        cout << "enter roll and marks";
        cin >> roll >> marks;
    }
    void showData() {
        cout << roll << "  " << marks << endl;
    }
};

int main()
{
    Student s1, s2;               // objects of the class
    s1.getData();
    s1.showData();
    return 0;
}
```

---

## ■ Data Members and Member Functions

The variables declared inside a class are its **data members**; the functions declared inside a class are its **member functions**. Member functions of three kinds: **normal member functions**, defined inside the class (automatically treated as inline) or outside with the **scope resolution operator ::**; **inline member functions**, asked-for by the keyword **inline** for short functions; and **static members**, shared by all objects of the class. A member function may also be **const**, which promises that it does not change any data member of the object that calls it.

```
class Square {
    int side;                    // private by default
public:
    void setSide(int s);         // declaration only
    int  area();
};

void Square::setSide(int s)      // definition outside the class
{
    side = s;
}

int Square::area()
{
    return side * side;
}
```

## ■ Encapsulation, Abstraction and the Basic Principles of OOP

- **Encapsulation** — the wrapping of data and functions into one single unit (the class) and keeping the data private: the capsule protects its contents; access happens only through the public member functions, so the data cannot be corrupted from outside.
- **Data abstraction** — showing only the necessary features and hiding the internal details: we use a car without knowing its engine; we call *cin.getline()* without knowing its code; the class gives an interface and hides the implementation.
- **Inheritance** — the making of a new class (derived) out of an existing class (base); the derived class receives the members of the base; it gives reusability of code.
- **Polymorphism** — one name, many forms: the same function name or operator behaves differently by the data it handles (function overloading, operator overloading, virtual functions).
- **Dynamic binding** — the linking of a function call to its body at run time (virtual functions); the code of the body is chosen when the program runs.
- **Message passing** — objects communicate by calling each other's member functions: object.function( ); is the sending of a message.

---

**Remember :** The four pillars of OOP in one look: **encapsulation**(data + functions in a sealed box), **abstraction**(interface without implementation), **inheritance**(new class from old class) and

**polymorphism**(one name, many forms). C++ supports all four, and is hence called an object-oriented, but not a purely object-oriented, language (unlike Java, C++ still permits plain procedural code and standalone functions).

---

# UNIT 3 — FUNCTIONS AND CONSTRUCTORS

## ■ Functions and Function Overloading

A **function** is a self-contained block with a name, a return type, a parameter list and a body; it is called by *name(arguments)*. Declaration (prototype), definition and call are the three parts of the function life in C++. Ways of passing parameters: **by value** (a copy is given; the original is safe), **by reference** (an alias is given with &; the original itself is changed) and **by pointer** (the address is passed with *). A function may also have **default arguments** (values used when the caller sends nothing) and may be **inline** (copied at the call site to save call overhead).

```
// call by reference
void swap(int &a, int &b)      // & gives the ACTUAL variables
{
    int temp = a;
    a = b;
    b = temp;
}

// default arguments
float simpleInterest(float p, float r, float t = 1.0)  // t optional
{
    return p * r * t / 100.0;
}

int main()
{
    int x = 3, y = 9;
    swap(x, y);                     // x=9, y=3 after the call
    cout << simpleInterest(1000, 5);        // t taken as 1
}
```

---

**Function overloading** means several functions with the **same name but different parameter lists** (different number, type or order of parameters); the compiler chooses the proper one by the arguments of the call at compilation time (static polymorphism). The return type alone may not make a difference, and a call must match exactly one version, otherwise it is an ambiguity error.

```
int    add(int a, int b)          { return a + b; }
double add(double a, double b)    { return a + b; }
int    add(int a, int b, int c)   { return a + b + c; }
```

```
add(2, 3);        // calls version 1
add(2.5, 3.5);    // calls version 2
add(1, 2, 3);     // calls version 3
```

## ■ Constructors: Purpose, Characteristics and Types

A **constructor** is a special member function with the **same name as the class, no return type** (not even void) which is called **automatically when an object is created**. Its purpose is to give initial values to the data members and to take every initial step (opening files, reserving memory). The characteristics: the name is fixed (the class name); it has no return type; it is always public; it may be overloaded; it cannot be called explicitly like an ordinary function and cannot be inherited or virtual; if we do not write any constructor the compiler gives the **default constructor** automatically.

| Type | How it looks | When it is called |
|---|---|---|
| Default constructor | Student() { roll = 0; } | when an object is made without any argument |
| Parameterised constructor | Student(int r, float m) { roll = r; marks = m; } | when arguments are given, e.g. Student s(5, 88.5); |
| Copy constructor | Student(const Student &s) { roll = s.roll; } | when one object is copied from another, e.g. Student s2(s1); |

---

**Remember :** A constructor may be overloaded (many constructors in one class, chosen by the arguments) but a destructor cannot. The copy constructor is needed particularly when the object contains pointers or dynamic memory; otherwise both objects would share the same memory.

## ■ Object Creation and Destructors

Dynamically-created objects of the class **Student**: **Student *s = new Student(5, 90);** makes the object on the heap with the parameterised constructor; **delete s;** destroys it and automatically calls its destructor. Objects may also be put in arrays **Student arr[3];** — then the default constructor runs for every element. The **destructor** is the special member with the class name preceded by a tilde (**~Student()**), with no return type and no parameter; it is called automatically when an object comes to the end of its life — when it goes out of scope or when **delete** is used. Its purpose is the last clean-up: releasing the memory, closing the files. There can only be **one** destructor in a class (never overloaded), and the destructors of derived objects run in the reverse order of the constructors.

```
class Demo {
public:
    Demo()    { cout << "constructor\n"; }   // born
    ~Demo()   { cout << "destructor\n"; }    // dies
};

void test()
{
```

```
    Demo d;              // constructor runs here
}                        // destructor runs automatically here

int main()
{
    test();
    Demo *p = new Demo;
    delete p;            // destructor runs here
    return 0;
}
```

---

# UNIT 4 — INHERITANCE AND POINTERS

## ■ Inheritance and its Types

**Inheritance** is the mechanism by which one class (the **derived** or child class) receives the properties and behaviour of another class (the **base** or parent class). It gives **reusability**: the already written and tested code of the base class need not be written again; and it is the natural base of **runtime polymorphism** through virtual functions. The syntax: **class Derived : visibility_mode Base { ... };** The visibility governs how the inherited members appear in the derived class: **public inheritance** keeps public members public and protected members protected; **private inheritance** makes everything private; **protected inheritance** keeps both protected. Private members of the base are never directly usable in the derived class, whatever the visibility.

| **Type of inheritance** | **Meaning** | **Syntax sketch** |
|---|---|---|
| Single | one base, one derived | class B : public A { }; |
| Multiple | one derived from two or more bases | class C : public A, public B { }; |
| Multilevel | class derived from a class which is itself derived | class B : public A { }; class C : public B { }; |
| Hierarchical | many derived classes from one base | class B : public A { }; class C : public A { }; |
| Hybrid | a combination of two or more of the above types — usually multiple + multilevel, which creates the diamond shape | class D : public B, public C { }; where B and C are both derived from A |

---

In **hybrid inheritance** the same base class is reachable by more than one path (the **diamond problem**): D gets two copies of the members of A. The cure is the **virtual base class**: *class B : virtual public A* and *class C : virtual public A*; then D receives only one shared copy of A. Constructor calls in inheritance run **from the base to the derived**, destructors in the reverse order; the derived class constructor may pass arguments to the base with the **initialiser list**: *Derived(int x) : Base(x) { }*.

## ■ The this Pointer

**this** is a pointer that every non-static member function receives automatically; it holds the **address of the object through which the function is called** (this of type Student*). Uses: to return the whole object from a member function ( **return *this;** in cascaded calls like s.set(5).set(6); ); to resolve a clash when a parameter has the same name as a data member (**this->roll = roll;**); and to pass the address of the current object to another function. **this** cannot be used in static member functions and it cannot be modified or deleted.

```
class Student {
    int roll;
public:
    Student& setRoll(int roll)
    {
        this->roll = roll;      // this->roll = the DATA MEMBER
        return *this;           // returns the object itself (cascading)
    }
};

Student s;
s.setRoll(10).setRoll(20);      // legal because setRoll returns *this
```

**Remember :** In simple words: **this** is the pointer to the object itself. Written inside a member function, *this->member* is exactly the same as writing *member* or *(*this).member*; it is used only when the same name covers two things or when the object itself has to be returned.

---

# UNIT 5 — TEMPLATES

## ■ Function Templates: Concept and Syntax

Problem solved by a template: add(int,int), add(float,float), add(double,double) all written by hand with overloading — repeated code. A **function template** writes the function **once with a generic type** and the compiler builds a separate version for every type used at call time (the process is called **template instantiation**; it happens at compilation time and is thus static polymorphism). The keyword is **template** with the **typename** (or class) placeholder: *template <typename T>*

```
template        // T = any type the caller sends
T add(T a, T b)
{
    return a + b;
}

int main()
{
    cout << add(4, 6);        // T = int    -> 10
```

```
    cout << add(2.5, 3.5);    // T = double -> 6
    cout << add(1.5f, 2.5f);  // T = float
    // cout << add(2, 3.5);   // ERROR: T cannot be both int and double
    return 0;
}
```

A template may take several placeholder types (**template <class T, class U>**), may be **overloaded**, and may also be given an **explicit type**: *add<int>(4, 6);*. The same idea for a whole class gives the **class template** (e.g. Stack of any type), from which STL containers such as *vector<int>* are made.

| **Function overloading** | **Function template** |
|---|---|
| Many functions are written by hand, one for every parameter list. | One generic function is written; the compiler makes the versions. |
| Each version may have entirely different code. | The same code is used for every type (operations must be valid for T). |
| The compiler selects among written functions. | The compiler generates a new function at every new type (instantiation). |
| Used when the logic differs. | Used when the logic is the same and only the type changes. |

---

## ■ Uses and Advantages of Generic Programming

- **Write once, use for every type** — no repeated code of int, float, double and char versions; fewer errors from copy-paste.
- **Type safety** — unlike C-style macros or void*, the template is checked by the compiler; wrong types are caught at compilation time.
- **Better maintainability** — one bug found is one bug fixed in the template, not in its ten copies.
- **Faster than run-time polymorphism** — the specialisation comes out at compilation time, so there is no virtual-call penalty.
- **Basis of the STL** — the whole C++ Standard Template Library (vector, list, map, sort, ...) is generic; the user sends his own type and gets a ready-made container or algorithm.
- **Code shrinking and reusability** in large projects: libraries can be written without knowing the user's data types in advance.

---

**Remember :** Remember: a template is not code by itself — it is a **recipe for code**. The compiler writes the real function the moment the template is used with a concrete type; therefore the body of the template must contain operations that are legal for every type used (e.g. <, +, ==).